rCore-Tutorial-v3 内核实现学习报告

概述

本报告基于 rCore-Tutorial-v3 教程的前六章内容，涵盖从裸机启动到文件系统的完整内核实现过程。

章节	主题
Ch1	应用程序与基本执行环境
Ch2	批处理系统
Ch3	多道程序与分时多任务
Ch4	地址空间
Ch5	进程
Ch6	文件系统

第一章：应用程序与基本执行环境

核心知识点

- **裸机编程**：没有操作系统支持，直接运行在硬件上
- **移除标准库依赖**：内核无法使用 `std`，只能使用 `core`
- **内核启动流程**：QEMU → RustSBI → 内核入口 → Rust main 函数
- **SBI 调用**：通过 `ecall` 指令调用 RustSBI 提供的服务（输出、关机等）
- **链接脚本**：控制程序的内存布局和入口点

核心实现

关键数据结构

```
// 本章主要涉及汇编级别的数据结构，无复杂 Rust struct
// 栈定义（在链接脚本中）
.globl boot_stack_top
boot_stack_bottom:
    .space 4096 * 16    # 64KB 内核栈
boot_stack_top:
```

核心接口

```
// SBI 调用封装 (sbi.rs)
pub fn sbi_call(eid: usize, fid: usize, arg0: usize, arg1: usize, arg2:
usize) -> usize {
    let mut ret;
```

```
    unsafe {
        asm!(
            "ecall",
            inlateout("x10") arg0 => ret,
            in("x11") arg1,
            in("x12") arg2,
            in("x16") fid,
            in("x17") eid,
        );
    }
    ret
}

// 字符输出
pub fn console_putchar(c: usize);

// 关机
pub fn shutdown() -> !;
```

汇编入口 (entry.asm)

```
.section .text.entry
.globl _start
_start:
    la sp, boot_stack_top    # 设置栈指针
    call rust_main           # 跳转到 Rust 主函数
```

第二章：批处理系统

核心知识点

- **特权级机制**：RISC-V 有 M（Machine）/S（Supervisor）/U（User）三个特权级
- **Trap 机制**：用户态↔内核态切换的硬件机制
- **系统调用**：用户程序通过 `ecall` 指令请求内核服务
- **批处理系统**：依次执行多个应用程序，每个程序独占 CPU
- **用户栈与内核栈**：不同特权级使用不同的栈

核心实现

关键数据结构

```
// Trap 上下文 (trap/context.rs)
#[repr(C)]
pub struct TrapContext {
    pub x: [usize; 32],        // 通用寄存器 x[0..31]
    pub sstatus: Sstatus,      // CSR sstatus 寄存器
    pub sepc: usize,           // CSR sepc 寄存器（异常返回地址）
```

```
}

impl TrapContext {
    // 设置栈指针
    pub fn set_sp(&mut self, sp: usize);

    // 初始化应用程序的 Trap 上下文
    pub fn app_init_context(entry: usize, sp: usize) -> Self;
}
```

核心接口

```
// Trap 处理入口 (trap.S)
// __alltraps: 保存用户态上下文，跳转到 trap_handler
// __restore: 恢复用户态上下文，通过 sret 返回

// Trap 处理器 (trap/mod.rs)
#[no_mangle]
pub fn trap_handler() -> !;

// 系统调用分发 (syscall/mod.rs)
pub fn syscall(syscall_id: usize, args: [usize; 3]) -> isize;

// 具体系统调用
pub fn sys_write(fd: usize, buf: *const u8, len: usize) -> isize;
pub fn sys_exit(exit_code: i32) -> !;
```

第三章：多道程序与分时多任务

核心知识点

- **多道程序**：在内存中同时存在多个程序
- **任务切换**：保存当前任务的上下文，恢复下一个任务的上下文
- **协作式调度**：任务主动调用 `yield` 让出 CPU
- **抢占式调度**：时钟中断强制切换任务
- **任务控制块（TCB）**：管理任务的所有信息

核心实现

关键数据结构

```
// 任务上下文 (task/task.rs)
// 仅包含 callee-saved 寄存器（调用者保存的寄存器由编译器自动处理）
#[derive(Clone, Copy)]
pub struct TaskContext {
    ra: usize,      // 返回地址
    sp: usize,      // 栈指针
```

```
    s: [usize; 12], // s0-s11 寄存器
}

// 任务控制块
pub struct TaskControlBlock {
    pub task_cx: TaskContext,    // 任务上下文
    pub task_status: TaskStatus,  // 任务状态
    pub pub_id: usize,            // 任务 ID
}

// 任务状态
pub enum TaskStatus {
    Ready,     // 就绪
    Running,   // 运行中
    Exited,    // 已退出
}
```

核心接口

```
// 任务切换 (task/switch.S)
// __switch(current_task_cx_ptr, next_task_cx_ptr)
pub unsafe extern "C" fn __switch(
    current_task_cx_ptr: *mut TaskContext,
    next_task_cx_ptr: *const TaskContext,
);

// 任务管理器 (task/manager.rs)
impl TaskManager {
    pub fn new() -> Self;
    pub fn add_task(&mut self, task: TaskControlBlock);
    pub fn fetch_task(&mut self) -> Option<&mut TaskControlBlock>;
}

// 任务调度
pub fn run_tasks();

// 系统调用
pub fn sys_yield() -> isize;  // 让出 CPU
```

第四章：地址空间

核心知识点

- **虚拟地址 vs 物理地址**：程序看到的地址 vs 实际硬件地址
- **页表**：虚拟地址到物理地址的映射表
- **SV39**：RISC-V 39 位虚拟地址模式，支持三级页表
- **地址空间**：一组有逻辑关联的虚拟内存区域（MapArea）
- **动态内存分配**：堆分配器的实现（基于 linked_list_allocator）

- **跳板机制**：解决内核/用户地址空间切换问题

核心实现

关键数据结构

```rust
// 地址类型 (mm/address.rs)
pub struct VirtAddr(usize);       // 虚拟地址
pub struct PhysAddr(usize);       // 物理地址
pub struct VirtPageNum(usize);    // 虚拟页号
pub struct PhysPageNum(usize);    // 物理页号

// 页表项 (mm/page_table.rs)
#[derive(Clone, Copy)]
pub struct PageTableEntry {
    pub bits: usize,  // [44:10] = PPN, [9:0] = flags
}

impl PageTableEntry {
    pub fn new(ppn: PhysPageNum, flags: PTEFlags) -> Self;
    pub fn ppn(&self) -> PhysPageNum;
    pub fn is_valid(&self) -> bool;
}

// 页表标志位
bitflags! {
    pub struct PTEFlags: u8 {
        const V = 1 << 0;  // 有效
        const R = 1 << 1;  // 可读
        const W = 1 << 2;  // 可写
        const X = 1 << 3;  // 可执行
        const U = 1 << 4;  // 用户态可访问
        const A = 1 << 6;  // 已访问
        const D = 1 << 7;  // 已修改
    }
}

// 页表
pub struct PageTable {
    root_ppn: PhysPageNum,  // 根页表的物理页号
}

// 内存区域 (mm/memory_set.rs)
pub struct MapArea {
    vpn_range: VPNRange,                            // 虚拟页号范围
    data_frames: BTreeMap<VirtPageNum, FrameTracker>,  // 物理帧
    map_type: MapType,                              // 映射类型
    map_perm: MapPermission,                        // 权限
}

pub enum MapType {
    Identical,  // 恒等映射
```

```
    Framed,     // 按帧分配
}

bitflags! {
    pub struct MapPermission: u8 {
        const R = 1 << 1;  // 可读
        const W = 1 << 2;  // 可写
        const X = 1 << 3;  // 可执行
        const U = 1 << 4;  // 用户态
    }
}

// 地址空间
pub struct MemorySet {
    page_table: PageTable,      // 页表
    areas: Vec<MapArea>,        // 内存区域列表
}
```

核心接口

```
// 页表管理
impl PageTable {
    pub fn new() -> Self;
    pub fn map(&mut self, vpn: VirtPageNum, ppn: PhysPageNum, flags:
PTEFlags);
    pub fn unmap(&mut self, vpn: VirtPageNum);
    pub fn find_pte(&self, vpn: VirtPageNum) -> Option<&mut
PageTableEntry>;
}

// 地址空间管理
impl MemorySet {
    pub fn new_empty() -> Self;
    pub fn new_kernel() -> Self;
    pub fn from_elf(elf_data: &[u8]) -> (Self, usize, usize);  // 从 ELF 创
建
    pub fn push(&mut self, area: MapArea, data: Option<&[u8]>);
    pub fn find_mut(&mut self, vpn: VirtPageNum) -> Option<&mut MapArea>;
}

// 物理帧分配器
pub trait FrameAllocator {
    fn alloc(&mut self) -> Option<PhysPageNum>;
    fn dealloc(&mut self, ppn: PhysPageNum);
}
```

第五章：进程

核心知识点

- **进程概念**：运行中的程序实例，包含独立的地址空间和资源
- **fork**：创建子进程，复制父进程的地址空间
- **exec**：用新程序替换当前进程的地址空间
- **waitpid**：等待子进程结束并回收资源
- **进程控制块（PCB）**：管理进程的所有信息（地址空间、文件描述符表、子进程等）
- **进程调度算法**：Stride 调度、多级反馈队列等

核心实现

关键数据结构

```
// 进程标识符 (task/id.rs)
pub struct PidHandle(pub usize);  // 进程 ID
pub struct TidHandle(pub usize);  // 线程 ID

// 进程控制块 (task/process.rs)
pub struct ProcessControlBlock {
    pub pid: PidHandle,  // 进程 ID
    inner: UPIntrFreeCell<ProcessControlBlockInner>,
}

pub struct ProcessControlBlockInner {
    pub is_zombie: bool,                                        // 是否为僵尸进
程
    pub memory_set: MemorySet,                                 // 地址空间
    pub parent: Option<Weak<ProcessControlBlock>>,              // 父进程
    pub children: Vec<Arc<ProcessControlBlock>>,                // 子进程列表
    pub exit_code: i32,                                         // 退出码
    pub fd_table: Vec<Option<Arc<dyn File + Send + Sync>>>,   // 文件描述符表
    pub tasks: Vec<Option<Arc<TaskControlBlock>>>,              // 线程列表
    pub task_res_allocator: RecycleAllocator,                   // 线程 ID 分
配器
}

// 任务控制块（关联到进程）
pub struct TaskControlBlock {
    pub process: Arc<ProcessControlBlock>,  // 所属进程
    pub task_res: Arc<TaskUserRes>,         // 用户资源（内核栈、Trap 上下文）
    pub task_cx: TaskContext,               // 任务上下文
}

// 任务用户资源
pub struct TaskUserRes {
    pub tid: TidHandle,              // 线程 ID
    pub ustack_base: usize,          // 用户栈基址
    pub trap_cx_ppn: PhysPageNum,    // Trap 上下文所在物理页
}
```

核心接口

```rust
// 进程相关系统调用 (syscall/process.rs)
pub fn sys_fork() -> isize;                          // 创建子进程
pub fn sys_exec(path: &str) -> isize;                // 执行新程序
pub fn sys_waitpid(pid: isize, exit_code: *mut i32) -> isize;  // 等待子进程
pub fn sys_exit(exit_code: i32) -> !;                // 退出进程

// 进程加载 (loader.rs)
pub fn load_app_from_disk(name: &str) -> (MemorySet, usize, usize);

// 进程管理器扩展
impl TaskManager {
    pub fn add_task(&mut self, process: Arc<ProcessControlBlock>);
    pub fn fetch_task(&mut self) -> Option<Arc<TaskControlBlock>>;
    pub fn exit_current_and_run_next(exit_code: i32);
}
```

第六章：文件系统

核心知识点

- **文件系统接口**：`open`/`close`/`read`/`write` 系统调用
- **Unix 文件模型**：一切皆文件，包括常规文件、目录、设备
- **easy-fs**：简易文件系统，基于 inode 的索引文件系统
- **inode**：索引节点，描述文件的元信息和数据块位置
- **文件描述符**：内核管理的文件引用，每个进程有独立的文件描述符表
- **块设备**：virtio-blk 模拟的块设备，按块（512 字节）读写
- **块缓存**：缓存最近访问的块，减少磁盘 I/O

核心实现

关键数据结构

```rust
// 文件 Trait (fs/mod.rs)
pub trait File: Send + Sync {
    fn readable(&self) -> bool;                  // 是否可读
    fn writable(&self) -> bool;                  // 是否可写
    fn read(&self, buf: UserBuffer) -> usize;    // 读取数据
    fn write(&self, buf: UserBuffer) -> usize;   // 写入数据
}

// 用户缓冲区（支持分散/聚集 I/O）
pub struct UserBuffer {
    pub buffers: Vec<&'static mut [u8]>,
}

// 打开标志
bitflags! {
    pub struct OpenFlags: u8 {
```

```rust
        const READONLY = 1 << 0;
        const WRITEONLY = 1 << 1;
        const READWRITE = 1 << 2;
        const CREATE = 1 << 3;      // 不存在则创建
        const TRUNC = 1 << 4;       // 截断文件
    }
}

// OSInode (fs/inode.rs)
pub struct OSInode {
    readable: bool,
    writable: bool,
    inner: Mutex<OSInodeInner>,
}

pub struct OSInodeInner {
    offset: usize,                      // 当前读写偏移
    inode: Arc<EasyFsInode>,            // 底层 easy-fs inode
}

// easy-fs 相关结构（在 easy-fs crate 中）
pub struct EasyFileSystem {
    pub block_device: Arc<dyn BlockDevice>,
    pub inode_disk: Arc<InodeDisk>,
}

pub struct Inode {
    pub block_id: usize,
    pub block_offset: usize,
    pub fs: Arc<EasyFileSystem>,
    pub node: InodeNode,
}

pub enum InodeNode {
    Direct,
    Indirect1,
    Indirect2,
}
```

核心接口

```rust
// 文件系统接口 (fs/inode.rs)
pub fn open_file(name: &str, flags: OpenFlags) -> Option<Arc<OSInode>>;
pub fn list_apps() -> Vec<String>;

// OSInode 实现
impl File for OSInode {
    fn read(&self, buf: UserBuffer) -> usize;
    fn write(&self, buf: UserBuffer) -> usize;
}
```

```rust
impl OSInode {
    pub fn new(readable: bool, writable: bool, inode: Arc<EasyFsInode>) ->
Self;
    pub fn clear(&self);  // 清空文件
}

// 文件系统相关系统调用 (syscall/fs.rs)
pub fn sys_openat(dir_fd: i32, path: &str, flags: u32, mode: u32) ->
isize;
pub fn sys_close(fd: usize) -> isize;
pub fn sys_read(fd: usize, buf: *const u8, len: usize) -> isize;
pub fn sys_write(fd: usize, buf: *const u8, len: usize) -> isize;
pub fn sys_fstat(fd: usize, stat_ptr: *mut u8) -> isize;

// 块设备接口（virtio-blk 驱动实现）
pub trait BlockDevice: Send + Sync {
    fn read_block(&self, block_id: usize, buf: &mut [u8]);
    fn write_block(&self, block_id: usize, buf: &[u8]);
}
```

总结

各章演进关系

```
Ch1: 裸机启动
  │
  ▼
Ch2: 批处理系统 ──► 加入特权级切换、系统调用
  │
  ▼
Ch3: 多道程序 ──► 加入任务切换、调度
  │
  ▼
Ch4: 地址空间 ──► 加入虚拟内存、页表
  │
  ▼
Ch5: 进程 ──► 加入 fork/exec/waitpid、进程管理
  │
  ▼
Ch6: 文件系统 ──► 加入 easy-fs、文件抽象
```

关键技术点

章节	关键技术
Ch1	SBI 调用、链接脚本、栈设置
Ch2	Trap 上下文、特权级切换、系统调用

章节	关键技术
Ch3	任务切换、协作/抢占式调度
Ch4	SV39 页表、地址空间、跳板机制
Ch5	进程控制块、fork/exec/waitpid
Ch6	inode 抽象、文件描述符、块缓存